Manual técnico

Informe 4

Jonathan Josué Girón Gómez

202403660

Index.js:

En este archivo se desarrolla la lógica para el servidor, utilizando el framework Express de Node.js para manejar las rutas hacia distintos endpoints para gestionar la base de datos.

Se crea una nueva aplicación Express mediante “const app = express()”. A partir de aquí, se utiliza “app.use()” para acceder a las funciones que procesan una petición (req) y una respuesta (res).

```
import express      from "express"

import userRoutes   from './routes/usuarios.routes.js'
import courseRoutes from './routes/cursos.routes.js'
import profRoutes   from './routes/catedraticos.routes.js'
import publiRoutes  from './routes/publicaciones.routes.js'
import comntRoutes  from './routes/comentarios.routes.js'
import aprCrsRoutes from './routes/cursos_apr.routes.js'

import indexRoutes  from './routes/index.routes.js'

const app = express()

app.use(express.json())

app.use(indexRoutes)
app.use('/api', userRoutes)
app.use('/api', courseRoutes)
app.use('/api', profRoutes)
app.use('/api', publiRoutes)
app.use('/api', comntRoutes)
app.use('/api', aprCrsRoutes)

app.listen(3000)
console.log("Server running on port 3000.")
```

Index.routes.js:

Aquí se realiza la prueba del servidor utilizando “http://localhost:3000/ping”. La función “ping” que utiliza la constante router se define en el controlador respectvo “index.controller.js”.

```
1   import { Router } from "express";
2   import { ping } from "../controllers/index.controller.js";
3
4   const router = Router()
5   router.get('/ping', ping)
6
7   export default router
```

Index.controller.js:

Aquí se define la función “ping”, el cual devuelve una respuesta para asegurarse de que existe conexión desde el backend hasta la base de datos.

```
import { pool } from "../db.js"

export const ping = async (req, res) => {
    const [result] = await pool.query('SELECT "Pong" AS result')
    res.json(result[0])
}
```

Usuarios.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar usuarios en la base de datos.

```
import { Router } from "express";
import { getUsers, getUser, createUser, updateUser, deleteUser } from '../controllers/usuarios.controller.js'

const router = Router()

router.get('/usuarios', getUsers)
router.get('/usuarios/:id_usuario', getUser)
router.post('/usuarios', createUser)
router.put('/usuarios/:id_usuario', updateUser)
router.delete('/usuarios/:id_usuario', deleteUser)

export default router
```

Usuarios.controller.js

Aquí se desarrollan las funciones para controlar los detalles de los usuarios. Mediante el uso de SELECT se puede simular el uso de un GET, mientras que POST se da al utilizar INSERT. Para actualizar los datos como un PUT, se utiliza UPDATE, y para eliminar datos se utiliza DELETE.

```
// Ver todos los usuarios
export const getUsers = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM usuario')
    res.json(rows)
}

// Ver un usuario
export const getUser = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM usuario WHERE id_usuario = ?', [req.params.id_usuario])
    if (rows.length <= 0) return res.status(404).json({
        message: "Usuario no encontrado."
    })
    res.json(rows[0])
}
```

```
// Crear un usuario
export const createUser = async (req, res) => {
    const {id_usuario, registro, nombre, apellido, correo, password} = req.body
    const [rows] = await pool.query('INSERT INTO usuario (id_usuario, registro, nombre, apellido, correo, password) VALUES (?, ?,
    [id_usuario, registro, nombre, apellido, correo, password], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_usuario, registro, nombre, apellido, correo, password})
}
```

```
// Actualizar un usuario
export const updateUser = async (req, res) => {
    const id_usuario = req.params.id_usuario
    const {nombre, registro, apellido, correo, password} = req.body
    const [rows] = await pool.query('UPDATE usuario SET registro = ?, nombre = ?, apellido = ?, correo = ?,password = ? WHERE id_u
    [registro, nombre, apellido, correo, password, id_usuario], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_usuario, nombre, registro, apellido, correo, password})
}


// Eliminar un usuario
export const deleteUser = async (req, res) => {
    const [result] = await pool.query('DELETE FROM usuario WHERE id_usuario = ?', [req.params.id_usuario])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "Usuario no encontrado."
    })
    res.sendStatus(204)
}
```

Cursos.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar cursos en la base de datos.

```
import { Router } from "express";
import { getCourses, getCourse, createCourse, updateCourse, deleteCourse } from '../controllers/cursos.controller.js'

const router = Router()

router.get('/cursos', getCourses)
router.get('/cursos/:id_curso', getCourse)
router.post('/cursos', createCourse)
router.put('/cursos/:id_curso', updateCourse)
router.delete('/cursos/:id_curso', deleteCourse)

export default router
```

Cursos.controller.js

Aquí se desarrollan las funciones para controlar los datos de los cursos. Al igual que con los usuarios, se utiliza SELECT para actuar como una opción GET, mientras que POST ocurre al utilizar INSERT. Para actualizar los datos como un PUT, se utiliza UPDATE, y para eliminar datos se utiliza DELETE.

```
// Ver todos los cursos
export const getCourses = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM CURSO')
    res.json(rows)
}

// Ver un curso
export const getCourse = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM CURSO WHERE id_curso = ?', [req.params.id_curso])
    if (rows.length <= 0) return res.status(404).json({
        message: "curso no encontrado."
    })
    res.json(rows[0])
}
```

```
// Crear un curso
export const createCourse = async (req, res) => {
    const {id_curso, nombre, creditos, area} = req.body
    const [rows] = await pool.query('INSERT INTO CURSO (id_curso, nombre, creditos, area) VALUES (?, ?, ?, ?)',
    [id_curso, nombre, creditos, area], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_curso, nombre, creditos, area})
}
```

```
// Actualizar un curso
export const updateCourse = async (req, res) => {
    const id_curso = req.params.id_curso
    const {nombre, creditos, area} = req.body
    const [rows] = await pool.query('UPDATE CURSO SET nombre = ?, creditos = ?, area = ? WHERE id_curso = ?',
    [nombre, creditos, area, id_curso], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_curso, nombre, creditos, area})
}


// Eliminar un curso
export const deleteCourse = async (req, res) => {
    const [result] = await pool.query('DELETE FROM CURSO WHERE id_curso = ?', [req.params.id_curso])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "curso no encontrado."
    })
    res.sendStatus(204)
}
```

Catedraticos.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar catedráticos en la base de datos.

```
import { Router } from "express";
import { getProfs, getProf, createProf, updateProf, deleteProf } from '../controllers/catedraticos.controller.js'

const router = Router()

router.get('/catedraticos', getProfs)
router.get('/catedraticos/:id_catedratico', getProf)
router.post('/catedraticos', createProf)
router.put('/catedraticos/:id_catedratico', updateProf)
router.delete('/catedraticos/:id_catedratico', deleteProf)

export default router
```

Catedraticos.controller.js:

Aquí se desarrollan las funciones para controlar los datos de los catedráticos. Se utiliza SELECT para actuar como GET, INSERT para actuar como POST, UPDATE para actuar como PUT, y DELETE para eliminar datos.

```
// Ver todos los catedraticos
export const getProfs = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM CATEDRATICO')
    res.json(rows)
}

// Ver un catedratico
export const getProf = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM CATEDRATICO WHERE id_catedratico = ?', [req.params.id_catedratico])
    if (rows.length <= 0) return res.status(404).json({
        message: "catedratico no encontrado."
    })
    res.json(rows[0])
}
```

```
// Crear un catedratico
export const createProf = async (req, res) => {
    const {id_catedratico, nombre, apellido, correo} = req.body
    const [rows] = await pool.query('INSERT INTO CATEDRATICO (id_catedratico, nombre, apellido, correo) VALUES (?, ?, ?, ?)',
    [id_catedratico, nombre, apellido, correo], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_catedratico, nombre, apellido, correo})
}
```

```
// Actualizar un catedratico
export const updateProf = async (req, res) => {
    const id_catedratico = req.params.id_catedratico
    const {nombre, apellido, correo} = req.body
    const [rows] = await pool.query('UPDATE CATEDRATICO SET nombre = ?, apellido = ?, correo = ? WHERE id_catedratico = ?',
    [nombre, apellido, correo, id_catedratico], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_catedratico, nombre, apellido, correo})
}


// Eliminar un catedratico
export const deleteProf = async (req, res) => {
    const [result] = await pool.query('DELETE FROM CATEDRATICO WHERE id_catedratico = ?', [req.params.id_catedratico])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "catedratico no encontrado."
    })
    res.sendStatus(204)
}
```

Publicaciones.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar publicaciones en la base de datos.

```
import { Router } from "express";
import { getPublis, getPubli, createPubli, updatePubli, deletePubli } from '../controllers/publicaciones.controller.js'

const router = Router()

router.get('/publicaciones', getPublis)
router.get('/publicaciones/:id_publicacion', getPubli)
router.post('/publicaciones', createPubli)
router.put('/publicaciones/:id_publicacion', updatePubli)
router.delete('/publicaciones/:id_publicacion', deletePubli)

export default router
```

Publicaciones.controller.js:

Aquí se desarrollan las funciones para controlar los datos de las publicaciones. Se utiliza SELECT para actuar como GET, INSERT para actuar como POST, UPDATE para actuar como PUT, y DELETE para eliminar datos. Además, cada modificación o agregación que se haga con respecto a una publicación debe tener referencias a un ID de usuario, un ID de curso y un ID de catedrático válidos.

```
// Ver todas las publicaciones
export const getPublis = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM PUBLICACION')
    res.json(rows)
}

// Ver una publicacion
export const getPubli = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM PUBLICACION WHERE id_publicacion = ?', [req.params.id_publicacion])
    if (rows.length <= 0) return res.status(404).json({
        message: "publicacion no encontrada."
    })
    res.json(rows[0])
}
```

```
// Crear una publicacion
export const createPubli = async (req, res) => {
    const {id_publicacion, mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso} = req.body
    const [rows] = await pool.query('INSERT INTO PUBLICACION (id_publicacion, mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_c
    [id_publicacion, mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_publicacion, mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso})
}
```

```
// Actualizar una publicacion
export const updatePubli = async (req, res) => {
    const id_publicacion = req.params.id_publicacion
    const {mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso} = req.body
    const [rows] = await pool.query('UPDATE PUBLICACION SET mensaje = ?, fecha = ?, USUARIO_id_usuario = ?, CATEDRATICO_id_catedr
    [mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso, id_publicacion], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_publicacion, mensaje, fecha, USUARIO_id_usuario, CATEDRATICO_id_catedratico, CURSO_id_curso})
}


// Eliminar una publicacion
export const deletePubli = async (req, res) => {
    const [result] = await pool.query('DELETE FROM PUBLICACION WHERE id_publicacion = ?', [req.params.id_publicacion])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "publicacion no encontrada."
    })
    res.sendStatus(204)
}
```

Comentarios.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar comentarios en la base de datos.

```
import { Router } from "express";
import { getComnts, getComnt, createComnt, updateComnt, deleteComnt } from '../controllers/comentarios.controller.js'

const router = Router()

router.get('/comentarios', getComnts)
router.get('/comentarios/:id_comentario', getComnt)
router.post('/comentarios', createComnt)
router.put('/comentarios/:id_comentario', updateComnt)
router.delete('/comentarios/:id_comentario', deleteComnt)

export default router
```

Comentarios.controller.js:

Aquí se desarrollan las funciones para controlar los datos de los comentarios. Se utiliza SELECT para actuar como GET, INSERT para actuar como POST, UPDATE para actuar como PUT, y DELETE para eliminar datos. Además, cada modificación o agregación que se haga con respecto a un comentario debe tener referencias a un ID de publicación y un ID de usuario válidos.

```
// Ver todos los comentarios
export const getComnts = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM COMENTARIO')
    res.json(rows)
}

// Ver un comentario
export const getComnt = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM COMENTARIO WHERE id_comentario = ?', [req.params.id_comentario])
    if (rows.length <= 0) return res.status(404).json({
        message: "comentario no encontrado."
    })
    res.json(rows[0])
}
```

```
// Crear un comentario
export const createComnt = async (req, res) => {
    const {id_comentario, mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario} = req.body
    const [rows] = await pool.query('INSERT INTO COMENTARIO (id_comentario, mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_ic
    [id_comentario, mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_comentario, mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario})
}
```

```
// Actualizar un comentario
export const updateComnt = async (req, res) => {
    const id_comentario = req.params.id_comentario
    const {mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario} = req.body
    const [rows] = await pool.query('UPDATE COMENTARIO SET mensaje = ?, fecha = ?, PUBLICACION_id_publicacion = ?, USUARIO_id_usua
    [mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario, id_comentario], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_comentario, mensaje, fecha, PUBLICACION_id_publicacion, USUARIO_id_usuario})
}


// Eliminar un comentario
export const deleteComnt = async (req, res) => {
    const [result] = await pool.query('DELETE FROM COMENTARIO WHERE id_comentario = ?', [req.params.id_comentario])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "comentario no encontrado."
    })
    res.sendStatus(204)
}
```

Cursos_apr.routes.js:

Se utiliza para controlar las funciones para ver, agregar, modificar y eliminar los cursos aprobados en la base de datos.

```
import { Router } from "express";
import { getAprCourses, getAprCourse, createAprCourse, updateAprCourse, deleteAprCourse } from '../controllers/cursos_apr.controll

const router = Router()

router.get('/cursosapr', getAprCourses)
router.get('/cursosapr/:id_curso', getAprCourse)
router.post('/cursosapr', createAprCourse)
router.put('/cursosapr/:id_curso', updateAprCourse)
router.delete('/cursosapr/:id_curso', deleteAprCourse)

export default router
```

Cursos_apr.controller.js:

Aquí se desarrollan las funciones para controlar los datos de los cursos. Se utiliza SELECT para actuar como GET, INSERT para actuar como POST, UPDATE para actuar como PUT, y DELETE para eliminar datos. Además, cada modificación o agregación que se haga con respecto a un comentario debe tener referencias a un ID de usuario y un ID de curso válidos.

```
// Ver todos los cursos aprobados
export const getAprCourses = async (req, res) => {
    const [rows] = await pool.query('SELECT * FROM CURSO_APR')
    res.json(rows)
}

// Ver un curso aprobado
export const getAprCourse = async (req, res) => {
    console.log(req.params)
    const [rows] = await pool.query('SELECT * FROM CURSO_APR WHERE id_curso_apr = ?', [req.params.id_curso])
    if (rows.length <= 0) return res.status(404).json({
        message: "curso no encontrado."
    })
    res.json(rows[0])
}
```

```
// Crear un curso aprobado
export const createAprCourse = async (req, res) => {
    const {id_curso_apr, fecha_apr, USUARIO_id_usuario, CURSO_id_curso} = req.body
    const [rows] = await pool.query('INSERT INTO CURSO_APR (id_curso_apr, fecha_apr, USUARIO_id_usuario, CURSO_id_curso) VALUES (?
    [id_curso_apr, fecha_apr, USUARIO_id_usuario, CURSO_id_curso], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_curso_apr, fecha_apr, USUARIO_id_usuario, CURSO_id_curso})
}
```

```
// Actualizar un curso aprobado
export const updateAprCourse = async (req, res) => {
    const id_curso_apr = req.params.id_curso
    const {fecha_apr, USUARIO_id_usuario, CURSO_id_curso} = req.body
    const [rows] = await pool.query('UPDATE CURSO_APR SET fecha_apr = ?, USUARIO_id_usuario = ?, CURSO_id_curso = ? WHERE id_curso
    [fecha_apr, USUARIO_id_usuario, CURSO_id_curso, id_curso_apr], (err, result) => {
        if (err){
            console.error("Error: "+err)
            return res.status(500).json(err)
        }
    })

    res.send({id_curso_apr, fecha_apr, USUARIO_id_usuario, CURSO_id_curso})
}


// Eliminar un curso aprobado
export const deleteAprCourse = async (req, res) => {
    const [result] = await pool.query('DELETE FROM CURSO_APR WHERE id_curso_apr = ?', [req.params.id_curso])
    console.log(result)
    if (result.affectedRows <= 0) return res.status(404).json({
        message: "curso no encontrado."
    })
    res.sendStatus(204)
}
```